Slacky Notes Architecture

Version 1.0 | 25 September 2026

Slacky Notes is a Windows and macOS desktop app that turns high-confidence task assignments from Slack direct messages into small, always-on-top sticky notes. It is designed for a free private beta of fewer than 1,000 users. The product favors precision over recall: an uncertain message must not become a task.

Product boundary

V1 supports Slack direct messages and group direct messages only. It creates a concise task with a permalink to its Slack source. Notes can be edited, completed, dismissed, and deleted locally. Users connect one active desktop installation to one Slack identity.

V1 does not support channels, mention scanning, inferred due dates, priorities, multi-device task sync, billing, app-store distribution, or storage of Slack message bodies.

System overview

The native desktop app is the user-facing task store and sticky-note renderer. A small cloud backend receives Slack events, decides whether a task exists, and temporarily holds an extracted task until the active device fetches it.

1. The user opens the app and authorizes Slack in the system browser.

2. The backend completes Slack OAuth, stores the encrypted Slack token, and binds the user to one device credential.

3. Slack sends a new direct-message event to the backend.

4. The backend verifies the Slack signature, rejects duplicate events, fetches a small bounded amount of recent context, and sends that transient input to OpenAI.

5. If OpenAI returns a valid, high-confidence task assigned to the connected Slack user, the backend creates an encrypted temporary outbox record containing only task text and source permalink.

6. The desktop app polls for pending records while it is running, stores a received task locally, displays the note, then acknowledges delivery. The backend deletes the outbox record.

Native desktop applications

macOS

Build the macOS app in Swift and SwiftUI. Use an `NSPanel` or equivalent non-activating window for the compact always-on-top sticky note. Store credentials and encryption keys in Keychain. Keep the local task database inside the app container.

Windows

Build the Windows app in C# with WinUI 3 and .NET. Use a topmost app window for the sticky note and Windows Credential Manager for device credentials and encryption keys. Keep the local task database in the app data directory.

Shared behavior contract

Both apps use the same behavior, even though their code is separate: launch and connect Slack; poll only while active; render outstanding notes above other windows; edit, complete, dismiss, and permanently delete local notes; show a quiet connection state; and retain completed or dismissed history until user deletion.

Backend architecture

Use TypeScript on Cloudflare Workers. Use Cloudflare D1 as the only V1 server-side database. This is intentionally a small architecture: it is sufficient for the private beta and avoids operating a dedicated server.

Worker responsibilities

- Run Slack OAuth start and callback routes.
- Verify Slack event request signatures before processing payloads.
- Deduplicate and checkpoint Slack events before any AI request.
- Retrieve only the bounded recent DM context necessary to interpret the new message.
- Call OpenAI using schema-constrained structured output.
- Encrypt and store Slack tokens, one device binding, operational checkpoints, and temporary outbox records in D1.
- Serve authenticated device registration, polling, and acknowledgement endpoints.

D1 records

- `slack_connection`: Slack user/workspace identifiers, encrypted OAuth token, connection status, and timestamps.
- `device_binding`: one active device ID, hashed device credential, platform, and last-seen time.
- `scan_checkpoint`: latest processed Slack event or message identifier per conversation and user.
- `event_deduplication`: bounded event identifiers used to make retries idempotent.
- `task_outbox`: encrypted task text, Slack permalink, task identifier, creation time, retry/delivery state, and expiration time.

No D1 table may contain a raw Slack message body, AI prompt text, or conversation transcript.

AI extraction contract

OpenAI receives only the newly arrived message, a small recent context slice, the Slack identity of the intended user, and an instruction to return JSON matching the defined schema. It must produce exactly one of these results:

- `no_task`: the message does not clearly assign an action to the user.
- `task`: concise imperative task text, confidence score, and the source permalink.

The backend accepts only schema-valid `task` results at or above a conservative configured confidence threshold. It discards invalid output, ambiguous assignments, explanations, summaries, and anything that is not a task. The model is not allowed to invent deadlines, priorities, or details absent from the message.

Interfaces

The backend exposes five small HTTP interfaces:

- Slack OAuth start and callback, which create or refresh the Slack connection.
- Slack event webhook, which handles URL verification and signed message events.
- Device registration, which exchanges a post-OAuth device challenge for a credential and replaces any prior active device.
- Pending-task polling, which returns the active device's undelivered task records.
- Delivery acknowledgement, which confirms a locally persisted task and deletes the temporary outbox copy.

The desktop app's local task model is: task ID, concise text, Slack permalink, status (`open`, `completed`, or `dismissed`), created time, modified time, and locally edited text. The backend model never receives edits, completion, or local history.

Privacy and security

- Use Slack OAuth rather than pasted personal tokens.
- Verify Slack signatures and protect OAuth state against CSRF.
- Encrypt Slack tokens and temporary outbox task content at rest; keep encryption key material outside D1 where Cloudflare-managed secrets can protect it.
- Store desktop credentials and database keys in the OS secure store.
- Send no raw Slack message body to D1, logs, analytics, or error reports.
- Redact request bodies and authorization values from logs.
- Use short-lived signed device credentials, HTTPS-only transport, rate limiting, and strict authorization checks on every device route.
- Let users disconnect Slack and delete their local data. Disconnecting revokes the Slack token, active device binding, checkpoints, and pending outbox records.

Reliability and failure behavior

Existing notes must remain usable when a dependency fails. Slack API errors, OpenAI errors, D1 errors, and device network failures use exponential backoff with jitter and expose only a non-blocking status indicator. Do not send repeated desktop alerts.

Slack event processing is idempotent. A failed AI call does not advance the successful processing checkpoint until it can be retried safely. Duplicate events cannot create duplicate notes. If the desktop app is offline, the backend retains only the encrypted extracted task and permalink until acknowledgement or expiry. If the device credential is invalid or another device becomes active, the app returns to the Slack connection flow.

Scale and operations

The V1 target is below 1,000 users. Cloudflare Workers and one D1 database are adequate if the app bounds context size, serializes or deduplicates event handling by conversation, and uses polling with a modest interval plus backoff. Track only non-content metrics: OAuth completion, event acceptance, model acceptance/rejection counts, delivery latency, retry counts, and error rates.

Before a paid release or material growth, reassess D1 throughput, outbox expiry, encryption/key rotation, provider cost, observability, multi-device policy, data deletion controls, and a managed queue if event volume warrants it.

Build phases

Phase 1 Foundation

Create the two native app shells, local task model, local storage, secure credential access, and a manually seeded sticky note. Acceptance: a task can be edited, completed, dismissed, and deleted on both platforms.

Phase 2 Slack connection

Create the Worker, D1 schema, Slack app configuration, OAuth flow, device registration, and disconnect flow. Acceptance: a Slack user can connect exactly one desktop app and revoke access.

Phase 3 Event to task pipeline

Implement verified Slack direct-message events, deduplication, checkpoints, bounded context retrieval, OpenAI structured extraction, confidence filtering, and the encrypted outbox. Acceptance: valid assignments create one outbox record; non-tasks and uncertain messages create none.

Phase 4 Desktop delivery

Implement authenticated polling, local persistence-before-acknowledgement, retries, outbox expiry, and connection status. Acceptance: a task received while the app is offline appears exactly once after reconnection.

Phase 5 Beta hardening

Add automated contract and integration tests, redacted operational metrics, installer signing, disconnect/delete controls, error states, and a private-beta feedback loop. Acceptance: no raw Slack

content is persisted or logged and failures do not lose or duplicate tasks.

Test plan

Test Slack signature verification, OAuth state validation, device replacement, expired credentials, event deduplication, checkpoint recovery, raw-message non-persistence, AI schema rejection, confidence threshold behavior, source permalink handling, outbox encryption, acknowledgement idempotency, offline recovery, local task lifecycle, and always-on-top behavior on both platforms.

Use adversarial messages during AI evaluation: requests meant for someone else, discussions without assignments, vague statements, multi-task messages, quoted text, and prompts attempting to alter extraction instructions. The required outcome is precision: the app must create a note only when a task for the connected user is clear.

Decision record

Accepted V1 decisions: native SwiftUI macOS app; native WinUI 3 Windows app; Slack DMs and group DMs only; Slack OAuth; Cloudflare Workers plus D1; OpenAI structured output; high-confidence automatic task creation; local-only task history; one active device; encrypted temporary cloud outbox; authenticated polling; direct signed installers; and free private beta.

Update this source document and regenerate the PDF whenever an accepted architecture decision changes.